

# Fast Parallel Image Rotation Algorithm

Dhrubajyoti Mandal  
School of Computer Engineering  
Kalinga Institute of Industrial Technology  
Bhubaneswar, India  
22053859@kiit.ac.in

Sourav Kumar Parida  
School of Computer Engineering  
Kalinga Institute of Industrial Technology  
Bhubaneswar, India  
22051032@kiit.ac.in

Subhadeep Bhadra  
School of Computer Engineering  
Kalinga Institute of Industrial Technology  
Bhubaneswar, India  
2205336@kiit.ac.in

Dr. Sujoy Dutta  
Asst. Prof. & Asst. CoE  
School of Computer Engineering  
Kalinga Institute of Industrial Technology  
Bhubaneswar, India  
sdattafcs@kiit.ac.in

Chiranjeev Bhaya  
Lead Engineer (Camera System)  
Samsung R&D Institute  
Bangalore, India  
c.bhaya@samsung.com

Chandra Mohan V  
Architect (Camera System)  
Samsung R&D Institute  
Bangalore, India  
chandra.mv@samsung.com

**Abstract**—This paper presents a novel fast image rotation algorithm that leverages CPU parallelization while maintaining high image quality. The proposed method is a single-pass algorithm, derived from the existing Double-Line Rotation (DLR) method, which reduces computational complexity and generalizes the algorithm. Initially, a baseline for the given image is calculated to determine the starting line, which defines the initial point for each vertical or horizontal line in the image to be rotated. The corresponding pixels to be mapped are identified using floating point arithmetic, and trigonometric calculations are performed only once per line. This approach ensures precise image transformation with minimal computational overhead.  
**Index terms:** Image rotation, line rotation image transform, double-line rotation (DLR), parallel image rotation.

## I. INTRODUCTION

2-D Image Rotation has a number of applications in digital photography and image processing like image matching, alignment, [?] etc. Highly accurate image rotation is essential for certain image processing tasks such as feature extraction and matching. Whilst there are many state-of-the-art image rotation algorithms, they have limitations in terms of image quality and speed [?] [?]. Parallel algorithms for image processing aim to execute faster than sequential algorithms, however designing and scheduling a fast parallel image rotation is a challenge [?]. The most straightforward, and perhaps the most well known approach to implement image rotation is by using the rotation matrix [?], as described here

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x - cx \\ y - cy \end{bmatrix} * \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

where,  $\theta$  is the angle of rotation,  $x'$  and  $y'$  are the new

coordinates of the rotated image, and  $(cx, cy)$  is the center of the original image, which is treated as the axis of rotation. Assuming the centre of rotation of the new image is  $(cx', cy')$ , the final new coordinates are:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x' + cx' \\ y' + cy' \end{bmatrix}$$

### A. Subsection Heading Here

Subsection text here.

1) Subsubsection Heading Here: Subsubsection text here.

## II. LITERATURE SURVEY

### A. Affine Transformation

A basic idea in computer graphics and geometry, affine transformation is a linear mapping method that maintains straight lines, planes, and points. The capacity to carry out a variety of geometric operations, including translation, scaling, rotation, shearing, and reflection, is what defines an Affine transformation. These transformations are powerful and adaptable tools with a wide range of applications because they are defined mathematically by combining translations and linear transformations.

#### 1) Properties:

- **Linearity and Translation:** The transformation is composed of a linear transformation  $Ax$  and a translation  $\beta$ . This combination allows for a wide range of geometric transformations while preserving the structure of the space.

- Preservation of Collinearity and Ratios: Affine transformations preserve the collinearity of points and the ratios of distances along parallel lines.
- Combining Transformations: Multiple affine transformations can be combined into a single affine transformation by matrix multiplication and vector addition.

#### MATHEMATICAL FORMULATION

Affine transformations can be described by a combination of linear transformations and translations. The general form of an affine transformation in two dimensions is:

$$y = Ax + b$$

where

- $x = \begin{bmatrix} x \\ y \end{bmatrix}$  is the input coordinate vector.
- $A$  is a  $2 \times 2$  matrix that represents the linear part of the transformation.
- $b = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$  is the translation vector.

For a rotation by an angle  $\theta$  around the origin, the matrix  $A$  is:

$$A = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

To center the rotated image within its original dimensions, the translation values  $t_x$  and  $t_y$  are calculated as follows:

#### CALCULATIONS

##### New Dimensions

The new width and height of the rotated image are determined using trigonometric functions based on the rotation angle:

$$newWidth = width \cdot |\cos(\theta)| + height \cdot |\sin(\theta)|$$

$$newHeight = width \cdot |\sin(\theta)| + height \cdot |\cos(\theta)|$$

##### Center Points

$$c_x = \frac{width}{2}, \quad c_y = \frac{height}{2}$$

$$newCx = \frac{newWidth}{2}, \quad newCy = \frac{newHeight}{2}$$

##### Translation Values

The translation values to center the rotated image within its new dimensions are:

$$t_x = c_x - (newCx \cdot \cos(\theta) - newCy \cdot \sin(\theta))$$

$$t_y = c_y - (newCx \cdot \sin(\theta) + newCy \cdot \cos(\theta))$$

Thus, the affine transformation matrix that includes rotation and translation is:

$$T = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & t_x \\ \sin(\theta) & \cos(\theta) & t_y \end{bmatrix}$$

The new coordinates  $(x', y')$  after applying the affine transformation are:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & t_x \\ \sin(\theta) & \cos(\theta) & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$x' = \cos(\theta) \cdot x - \sin(\theta) \cdot y + t_x$$

$$y' = \sin(\theta) \cdot x + \cos(\theta) \cdot y + t_y$$

#### B. Affine Transformation Algorithm

The following algorithm demonstrates how to rotate an image by a specified angle using OpenCV:

- Get image dimensions (rows and cols)
- Convert the angle from degrees to radians
- Calculate new dimensions for the rotated image (new\_cols, new\_rows)
- Create a new image (rotated) with the new dimensions, initialized to black

---

#### Algorithm 1 Image Rotation using Affine Transformation

---

```

1: procedure ROTATE_AFFINE(...)
2:   Compute  $\alpha = \cos(\text{angle})$  and  $\beta = \sin(\text{angle})$ 
3:   Compute the center coordinates of the original and
      new images:
4:    $c_x = \text{cols}/2.0$ 
5:    $c_y = \text{rows}/2.0$ 
6:    $new\_c\_x = new\_cols/2.0$ 
7:    $new\_c\_y = new\_rows/2.0$ 
8:   Define the rotation matrix elements:
9:    $m_{11} = \alpha$ 
10:   $m_{12} = \beta$ 
11:   $m_{13} = (1-\alpha) \cdot new\_c\_x - \beta \cdot new\_c\_x + c_x - new\_c\_x$ 
12:   $m_{21} = -\beta$ 
13:   $m_{22} = \alpha$ 
14:   $m_{23} = \beta \cdot new\_c\_x + (1-\alpha) \cdot new\_c\_y + c_y - new\_c\_y$ 
15:  for each pixel  $y$  from 0 to new_rows - 1 do
16:    for each pixel  $x$  from 0 to new_cols - 1 do
17:      Compute the corresponding coordinates
       $(x_, y_)$  in the original image:
18:       $x_ = \text{round}(m_{11} \cdot x + m_{12} \cdot y + m_{13})$ 
19:       $y_ = \text{round}(m_{21} \cdot x + m_{22} \cdot y + m_{23})$ 
20:      if  $(x_, y_)$  is within the bounds of the original
      image then
21:        Set the pixel value at  $(x, y)$  in the new
        image to the value at  $(x_, y_)$  in the original image
22:      end if
23:    end for
24:  end for
25:  return rotated
26: end procedure

```

---

### III. IMAGE ROTATION USING HERMITE EXPANSION

#### A. Hermite Expansion Method

The Hermite polynomial expansion is a technique used to approximate functions, including image rotation. By using

Hermite polynomials, we can achieve smooth and accurate interpolation, which is especially useful for image processing tasks.

#### 1) Properties of Hermite Polynomials:

- Orthogonal polynomials with respect to the weight function  $e^{-x^2}$ .
- Useful for approximating smooth functions due to their interpolation properties.
- Can handle both function values and derivatives, providing higher accuracy.

#### B. Mathematical Formulation

Hermite polynomials, denoted as  $H_n(x)$ , are a set of orthogonal polynomials that arise in probability, combinatorics, and numerical analysis. They are defined by the recurrence relation:

$$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x),$$

with initial conditions  $H_0(x) = 1$  and  $H_1(x) = 2x$ . These polynomials satisfy the orthogonality condition:

$$\int_{-\infty}^{\infty} H_m(x)H_n(x)e^{-x^2}dx$$

#### C. Hermite Expansion

A function  $f(x)$  can be expanded in terms of Hermite polynomials as:

$$f(x) = \sum_{n=0}^{\infty} c_n H_n(x) e^{-x^2/2},$$

where  $c_n$  are the expansion coefficients given by:

$$c_n = \frac{1}{\sqrt{2^n n! \sqrt{\pi}}} \int_{-\infty}^{\infty} f(x) H_n(x) e^{-x^2/2} dx.$$

#### D. Application of Hermite Expansion in Image Rotation

The use of Hermite expansion in image rotation leverages the orthogonality of Hermite polynomials to approximate the image function. Given an image  $I(x, y)$ , it can be expanded using two-dimensional Hermite polynomials  $H_{m,n}(x, y)$  as:

$$I(x, y) = \sum_{m=0}^{\infty} \sum_{n=0}^{\infty} c_{m,n} H_m(x) H_n(y) e^{-(x^2+y^2)/2},$$

where  $c_{m,n}$  are the expansion coefficients. These coefficients are computed as:

$$c_{m,n} = \frac{1}{\sqrt{2^{m+n} m! n! \pi}} \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} I(x, y) H_m(x) H_n(y) e^{-(x^2+y^2)/2} dx dy.$$

To rotate the image, the coordinates  $(x, y)$  are transformed to  $(x', y')$  as previously defined:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

1) *Finding Transformed Coordinates Using Hermite Expansion:* To find the transformed coordinates  $(x', y')$  using Hermite expansion, we perform the following steps:

Expand the Image Function Using Hermite Polynomials: Given an image  $I(x, y)$ , we first expand it using the two-dimensional Hermite polynomials  $H_{m,n}(x, y)$ :

$$I(x, y) = \sum_{m=0}^{\infty} \sum_{n=0}^{\infty} c_{m,n} H_m(x) H_n(y) e^{-(x^2+y^2)/2}$$

where the coefficients  $c_{m,n}$  are determined using:

$$c_{m,n} = \frac{1}{\sqrt{2^{m+n} m! n! \pi}} \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} I(x, y) H_m(x) H_n(y) e^{-(x^2+y^2)/2} dx dy.$$

Compute the Rotated Coordinates: Using the rotation matrix, we transform the coordinates  $(x, y)$  to  $(x', y')$ :

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

Reconstruct the Rotated Image: The rotated image  $I'(x', y')$  is reconstructed using the inverse Hermite expansion:

$$I'(x', y') = \sum_{m=0}^{\infty} \sum_{n=0}^{\infty} c_{m,n} H_m(x') H_n(y') e^{-(x'^2+y'^2)/2},$$

where  $H_m(x')$  and  $H_n(y')$  are the Hermite polynomials evaluated at the transformed coordinates  $(x', y')$ .

By following these steps, the image can be rotated accurately using the Hermite expansion. This method ensures that the transformation leverages the orthogonality and interpolation properties of Hermite polynomials, resulting in a smooth and precise image rotation.

### IV. IMAGE ROTATION USING DOUBLE LINE ROTATION

The Double Line Rotation (DLR) method is an innovative algorithm for fast image rotation, introduced to address the inefficiencies in traditional rotation techniques. The DLR method is based on the concept of dividing the image into multiple zones, each processed differently to achieve an efficient and accurate rotation.

#### A. Methodology

The DLR method divides the image into eight distinct zones, each characterized by specific rotation equations and transformations. The main idea is to map each pixel from the original image to two corresponding pixels in the target image, ensuring that the distance ratio between points remains constant throughout the rotation process. This is particularly useful in preserving the intensity and shape of the pixels, making the DLR method suitable for applications requiring invariant feature extraction.

#### B. Zone Concepts

The image is divided into the following zones:

1) *Zone 1*: In Zone 1, horizontal lines are used for rotation, and the slope of  $f_s(x)$  is positive. Each horizontal line in the original image  $T$  lies in a line with slope  $\tan(\alpha)$  and shears using the line  $f_s(x) = (\tan \alpha + \frac{\pi}{2})x$ . The size of the rotated image is calculated using:

$$m_{rt} = \gamma_m + m, \quad n_{rt} = \gamma_n + n$$

where  $\gamma_n = f_s(m) \rightarrow \left\| \frac{m}{\tan(\alpha + \frac{\pi}{2})} \right\|$  and  $\gamma_m = f(n) \rightarrow \|n \tan \alpha\|$ .

2) *Zone 2*: In Zone 2, vertical lines are used for rotation, and the slope of  $f(y)$  is positive. The size of the rotated image is calculated similarly to Zone 1, but the vertical and horizontal lines are interchanged.

3) *Zone 3*: In Zone 3, the slope is negative, but the equations are the same as for Zone 2. Vertical lines are used, and the size of the rotated image  $R(m_{rt} \times n_{rt})$  is calculated as follows:

$$m_{rt} = \gamma_m + n, \quad n_{rt} = \gamma_n + m$$

where  $\gamma_n = |f_s(n)| \rightarrow f_s(y) = \frac{y}{-\tan \alpha}$  and  $\gamma_m = \sqrt{m^2 - AC^2}$  in  $\triangle ABC$ .

4) *Zone 4*: In Zone 4, horizontal lines are used, and the slope of  $f_s(x)$  is positive, but  $f(x)$  has a negative gradient. This means that lines are moving upwards, the original image's lines are sloping downwards. Each horizontal line in image  $T$  is mapped and similarly to Zone 1. The involves altering the position of each pixel along the horizontal lines according to the function

$$f_s(x) = \left( \tan \alpha + \frac{\pi}{2} \right) x.$$

5) *Zone 5*: Zone 5 employs logical transformations on indices to generate the rotated image. These transformations involve manipulation of pixel positions such as swaps, inversionsof indices. The goal is to create a rotated image that retains the original dimensions while applying a different logical structure to the pixel arrangement. All zones are transformed, depending on whether vertical or horizontal lines are used. The size of the rotated image in Zone 1 and Zone 5 are the same, ensuring consistency in image dimensions.

6) *Zone 6*: Zone 6 is generated by transforming either Zone 2 or Zone 3. The transformation maintains the size of the rotated image and applies internal line transformations. These internal line transformations may include skewing, scaling, or shifting along specific axes to emphasize certain internal features.

7) *Zone 7*: Zone 7, similar to Zone 6, is a transformation of other zones but uses vertical lines. The vertical line transformations involve modifying the image along the vertical axis, affecting the height of the image without altering its width. This vertical stretching or compressing highlights vertical patterns and structures within the image.

8) *Zone 8*: Zone 8 utilizes transformations from horizontal lines, similar to Zones 1 and 4. These horizontal transformations involve scaling operations that modify the image along the horizontal axis. The transformations in Zone 8 ensure that

the overall balance of the image is maintained while providing a comprehensive view of horizontally oriented features.

### C. Calculation of the Base-Line, Start-Line, and Target Image Size

The first step in the DLR method is calculating the target image size. Each zone uses specific equations to determine the size of the target image, ensuring accurate and efficient rotation. For example, in Zone 3:

$$\gamma_n = |f_s(n)| \rightarrow f_s(y) = \frac{y}{-\tan \alpha}, \quad \gamma_m = \sqrt{m^2 - AC^2}$$

where  $AC = (\cos \phi)m$  and  $\phi = (\pi - \alpha) - \frac{\pi}{2}$ .

By dividing the image into zones and applying the DLR method, the rotation process becomes more efficient and accurate, preserving the original image's intensity and shape.

## V. CONCLUSION

The conclusion goes here.

## ACKNOWLEDGMENT

The authors would like to thank...